

SQL Befehle

Tabelle Erstellen

```
CREATE TABLE Teilnehmer (  
  TeilnehmerID INT PRIMARY KEY AUTO_INCREMENT,  
  Vorname VARCHAR(50) NOT NULL,  
  Nachname VARCHAR(50) NOT NULL,  
  Geburtsdatum DATE,  
  Email VARCHAR(100) UNIQUE,  
  Registrierungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Erstellt die Tabelle Teilnehmer mit Feldern für **ID**, **Name**, **Geburtsdatum**, **Email** und **Registrierungsdatum**.

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email	Registrierungsdatum
INT	VARCHAR(50)	VARCHAR(50)	DATE	VARCHAR(100) (UNIQUE)	TIMESTAMP (DEFAULT CURRENT_TIMESTAMP)

Relationen Bauen

```
CREATE TABLE Adresse (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  Stadt VARCHAR(100) NOT NULL,  
  Vorwahl VARCHAR(10) NOT NULL,  
  Straße VARCHAR(150) NOT NULL,  
  TeilnehmerID INT,  
  FOREIGN KEY (TeilnehmerID) REFERENCES Teilnehmer(TeilnehmerID)  
);
```

Erstellt die Tabelle Adresse mit Feldern für **ID**, **Stadt**, **Vorwahl**, **Straße** und eine **TeilnehmerID**, die auf die Teilnehmer-Tabelle verweist.

id	Stadt	Vorwahl	Straße	TeilnehmerID
INT	VARCHAR(100)	VARCHAR(10)	VARCHAR(150)	INT

Neue Spalte Einfügen

```
ALTER TABLE Adresse  
ADD COLUMN Hausnummer INT NOT NULL,  
ADD COLUMN Etage INT NOT NULL;
```

Fügt der Adresse-Tabelle eine **neue Spalte** Hausnummer hinzu, die einen Wert vom Typ VARCHAR(10) erwartet und nicht leer sein darf.

id	Stadt	Vorwahl	Straße	TeilnehmerID	Hausnummer	Etage
INT	VARCHAR(100)	VARCHAR(10)	VARCHAR(150)	INT	INT	INT

Eine Spalte löschen

```
ALTER TABLE Adresse
DROP COLUMN Etage;
```

id	Stadt	Vorwahl	Straße	TeilnehmerID	Hausnummer
INT	VARCHAR(100)	VARCHAR(10)	VARCHAR(150)	INT	INT

Eine Spalte modifizieren

```
ALTER TABLE Adresse
MODIFY COLUMN Hausnummer VARCHAR(50) NOT NULL;
```

id	Stadt	Vorwahl	Straße	TeilnehmerID	Hausnummer
INT	VARCHAR(100)	VARCHAR(10)	VARCHAR(150)	INT	VARCHAR(50)

Add Teilnehmer

```
INSERT INTO Teilnehmer (Vorname, Nachname, Geburtsdatum, Email)
VALUES
  ('Max', 'Mustermann', '1990-05-15', 'max.mustermann@example.com'),
  ('Anna', 'Schmidt', '1985-08-22', 'anna.schmidt@example.com'),
  ('Tom', 'Müller', '1995-03-10', 'tom.mueller@example.com'),
  ('Lisa', 'Fischer', '1992-07-30', 'lisa.fischer@example.com'),
  ('Paul', 'Wagner', '1988-11-12', 'paul.wagner@example.com'),
  ('Julia', 'Becker', '1994-02-25', 'julia.becker@example.com'),
  ('Felix', 'Hoffmann', '1991-09-18', 'felix.hoffmann@example.com'),
  ('Sarah', 'Schulz', '1987-04-05', 'sarah.schulz@example.com'),
  ('David', 'Koch', '1993-12-20', 'david.koch@example.com'),
  ('Laura', 'Bauer', '1996-06-08', 'laura.bauer@example.com');
```

Fügt zehn neue Teilnehmer mit Vornamen, Nachnamen, Geburtsdaten und E-Mail-Adressen in die Teilnehmer-Tabelle ein.

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email	Registrierungsdatum
1	Max	Mustermann	1990-05-15	max.mustermann@example.com	(automatisch gesetzt)
2	Anna	Schmidt	1985-08-22	anna.schmidt@example.com	(automatisch gesetzt)
3	Tom	Müller	1995-03-10	tom.mueller@example.com	(automatisch gesetzt)
4	Lisa	Fischer	1992-07-30	lisa.fischer@example.com	(automatisch gesetzt)
5	Paul	Wagner	1988-11-12	paul.wagner@example.com	(automatisch gesetzt)
6	Julia	Becker	1994-02-25	julia.becker@example.com	(automatisch gesetzt)
7	Felix	Hoffmann	1991-09-18	felix.hoffmann@example.com	(automatisch gesetzt)
8	Sarah	Schulz	1987-04-05	sarah.schulz@example.com	(automatisch gesetzt)
9	David	Koch	1993-12-20	david.koch@example.com	(automatisch gesetzt)
10	Laura	Bauer	1996-06-08	laura.bauer@example.com	(automatisch gesetzt)

Add Adresse

```
INSERT INTO Adresse (Stadt, Vorwahl, Straße, Hausnummer, TeilnehmerID)
VALUES
  ('Berlin', '030', 'Hauptstraße', '15', 1),
  ('Berlin', '089', 'Nebenstraße', '22A', 2),
  ('Berlin', '040', 'Bahnhofstraße', '5', 3),
  ('Frankfurt', '0221', 'Rathausstraße', '10', 4),
  ('Frankfurt', '069', 'Parkstraße', '8B', 5),
  ('Frankfurt', '0711', 'Lindenstraße', '3', 6),
  ('Berlin', '0211', 'Friedrichstraße', '20', 7),
  ('Bremen', '0341', 'Goethestraße', '12', 8),
  ('Bremen', '0351', 'Altstadtstraße', '4', 9),
  ('Bremen', '0421', 'Weserstraße', '7', 10);
```

Fügt zehn neue Adressen mit Stadt, Vorwahl, Straße, Hausnummer und der TeilnehmerID in die Adresse-Tabelle ein, wobei jede Adresse einem Teilnehmer zugeordnet wird.

id	Stadt	Vorwahl	Straße	Hausnummer	TeilnehmerID
1	Berlin	030	Hauptstraße	15	1
2	Berlin	089	Nebenstraße	22A	2
3	Berlin	040	Bahnhofstraße	5	3
4	Frankfurt	0221	Rathausstraße	10	4
5	Frankfurt	069	Parkstraße	8B	5
6	Frankfurt	0711	Lindenstraße	3	6
7	Berlin	0211	Friedrichstraße	20	7
8	Bremen	0341	Goethestraße	12	8
9	Bremen	0351	Altstadtstraße	4	9
10	Bremen	0421	Weserstraße	7	10

Select all

```
SELECT * FROM Teilnehmer;
```

Gibt **alle** Spalten und Zeilen aus der Teilnehmer-Tabelle zurück

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email	Registrierungsdatum
1	Max	Mustermann	1990-05-15	max.mustermann@example.com	(automatisch gesetzt)
2	Anna	Schmidt	1985-08-22	anna.schmidt@example.com	(automatisch gesetzt)
3	Tom	Müller	1995-03-10	tom.mueller@example.com	(automatisch gesetzt)
4	Lisa	Fischer	1992-07-30	lisa.fischer@example.com	(automatisch gesetzt)
5	Paul	Wagner	1988-11-12	paul.wagner@example.com	(automatisch gesetzt)
6	Julia	Becker	1994-02-25	julia.becker@example.com	(automatisch gesetzt)

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email	Registrierungsdatum
7	Felix	Hoffmann	1991-09-18	felix.hoffmann@example.com	(automatisch gesetzt)
8	Sarah	Schulz	1987-04-05	sarah.schulz@example.com	(automatisch gesetzt)
9	David	Koch	1993-12-20	david.koch@example.com	(automatisch gesetzt)
10	Laura	Bauer	1996-06-08	laura.bauer@example.com	(automatisch gesetzt)

Select bestimmte Spalten

```
SELECT Email FROM Teilnehmer;
```

Gibt nur die Email-Spalte aus der Teilnehmer-Tabelle zurück.

Email
max.mustermann@example.com
anna.schmidt@example.com
tom.mueller@example.com
lisa.fischer@example.com
paul.wagner@example.com
julia.becker@example.com
felix.hoffmann@example.com
sarah.schulz@example.com
david.koch@example.com
laura.bauer@example.com

```
SELECT Hausnummer FROM Adresse;
```

Gibt nur die Hausnummer-Spalte aus der Adresse-Tabelle zurück.

Hausnummer
15
22A
5
10
8B
3
20
12
4
7

```
SELECT TeilnehmerID, Straße FROM Adresse;
```

Gibt die TeilnehmerID und Straße-Spalten aus der Adresse-Tabelle zurück.

TeilnehmerID	Straße
1	Hauptstraße
2	Nebenstraße
3	Bahnhofstraße
4	Rathausstraße
5	Parkstraße
6	Lindenstraße
7	Friedrichstraße
8	Goethestraße
9	Altstadtstraße
10	Weserstraße

Select Distinct

```
SELECT DISTINCT Stadt FROM Adresse;
```

Gibt die **einzigartigen** Werte (keine Duplikate) der Stadt-Spalte aus der Adresse-Tabelle zurück.

Stadt
Berlin
Frankfurt
Bremen

Select where (select mit Voraussetzungen)

```
SELECT Vorname, Nachname, Geburtsdatum, Email
FROM Teilnehmer
WHERE Email = 'max.mustermann@example.com';
```

Gibt die Vorname, Nachname, Geburtsdatum und Email des Teilnehmers mit der Email-Adresse 'max.mustermann@example.com' aus der Teilnehmer-Tabelle zurück.

Vorname	Nachname	Geburtsdatum	Email
Max	Mustermann	1990-05-15	max.mustermann@example.com

```
SELECT Vorname, Nachname, Registrierungsdatum
FROM Teilnehmer
WHERE Geburtsdatum > '1990-01-01';
```

Gibt die Vorname, Nachname und Registrierungsdatum der Teilnehmer zurück, deren Geburtsdatum nach dem 01.01.1990 liegt.

Vorname	Nachname	Registrierungsdatum
Anna	Schmidt	(automatisch gesetzt)
Lisa	Fischer	(automatisch gesetzt)
Julia	Becker	(automatisch gesetzt)
Felix	Hoffmann	(automatisch gesetzt)
David	Koch	(automatisch gesetzt)
Laura	Bauer	(automatisch gesetzt)

```
SELECT Straße, Hausnummer, Vorwahl
FROM Adresse
WHERE Stadt = 'Berlin';
```

Gibt die Straße, Hausnummer und Vorwahl der Teilnehmer aus der Adresse-Tabelle zurück, bei denen die Stadt 'Berlin' ist.

Straße	Hausnummer	Vorwahl
Hauptstraße	15	030
Nebenstraße	22A	089
Bahnhofstraße	5	040
Friedrichstraße	3	0711

Select where mit WHERE EXIST

```
SELECT t.*
FROM Teilnehmer t
WHERE EXISTS (
    SELECT 1
    FROM Adresse a
    WHERE a.Stadt = 'Berlin'
);
```

WHERE EXISTS ist eine boolean Syntax, anzeigt die alle Zeilen, wenn einer Zeile die Voraussetzung nach WHERE erfüllt, sonst anzeigt nichts.

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email	Registrierungsdatum
1	Max	Mustermann	1990-05-15	max.mustermann@example.com	(automatisch gesetzt)
2	Anna	Schmidt	1985-08-22	anna.schmidt@example.com	(automatisch gesetzt)
3	Tom	Müller	1995-03-10	tom.mueller@example.com	(automatisch gesetzt)
4	Lisa	Fischer	1992-07-30	lisa.fischer@example.com	(automatisch gesetzt)
5	Paul	Wagner	1988-11-12	paul.wagner@example.com	(automatisch gesetzt)
6	Julia	Becker	1994-02-25	julia.becker@example.com	(automatisch gesetzt)

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email	Registrierungsdatum
7	Felix	Hoffmann	1991-09-18	felix.hoffmann@example.com	(automatisch gesetzt)
8	Sarah	Schulz	1987-04-05	sarah.schulz@example.com	(automatisch gesetzt)
9	David	Koch	1993-12-20	david.koch@example.com	(automatisch gesetzt)
10	Laura	Bauer	1996-06-08	laura.bauer@example.com	(automatisch gesetzt)

hinweis : WHERE NOT EXIST funktioniert genau umgekehrt

Select where mit IN

```
SELECT *
FROM Teilnehmer
WHERE TeilnehmerID IN (
    SELECT TeilnehmerID
    FROM Adresse
    WHERE Stadt = 'Berlin'
);
```

WHERE IN ist eine boolean Syntax, anzeigt nur die Zeilen, die die Voraussetzung nach WHERE erfüllt.

TeilnehmerID	Vorname	Nachname	Geburtsdatum
1	Max	Mustermann	1990-05-15
4	Anna	Schmidt	1992-07-30
8	Tom	Müller	1987-04-05

Hinwes: WHERE NOT IN funktioniert genau wie WHERE IN aber umgekehrt.

Neue teinehmer Einfügen

```
INSERT INTO Teilnehmer (Vorname, Nachname, Geburtsdatum, Email)
VALUES ('Michael', 'Weber', '1998-12-05', 'michael.weber@example.com');
```

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email
1	Max	Mustermann	1990-05-15	max.mustermann@example.com
2	Anna	Schmidt	1985-08-22	anna.schmidt@example.com
3	Tom	Müller	1995-03-10	tom.mueller@example.com
4	Lisa	Fischer	1992-07-30	lisa.fischer@example.com
5	Paul	Wagner	1988-11-12	paul.wagner@example.com
6	Julia	Becker	1994-02-25	julia.becker@example.com
7	Felix	Hoffmann	1991-09-18	felix.hoffmann@example.com
8	Sarah	Schulz	1987-04-05	sarah.schulz@example.com
9	David	Koch	1993-12-20	david.koch@example.com

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email
10	Laura	Bauer	1996-06-08	laura.bauer@example.com
11	Michael	Weber	1998-12-05	michael.weber@example.com

INNER JOIN

```
SELECT Teilnehmer.Vorname, Teilnehmer.Nachname, Adresse.Straße
FROM Teilnehmer
INNER JOIN Adresse ON Teilnehmer.TeilnehmerID = Adresse.TeilnehmerID;
```

Gibt die Vornamen, Nachnamen und Straßennamen aller Teilnehmer zurück, die eine Adresse haben. **Teilnehmer ohne Adresse werden ausgeschlossen.**

Vorname	Nachname	Straße
Max	Mustermann	Hauptstraße
Anna	Schmidt	Nebenstraße
Tom	Müller	Bahnhofstraße
Lisa	Fischer	Rathausstraße
Paul	Wagner	Parkstraße
Julia	Becker	Lindenstraße
Felix	Hoffmann	Friedrichstraße
Sarah	Schulz	Goethestraße
David	Koch	Altstadtstraße
Laura	Bauer	Weserstraße

LEFT JOIN

```
SELECT Teilnehmer.Vorname, Teilnehmer.Nachname, Adresse.Straße
FROM Teilnehmer
LEFT JOIN Adresse ON Teilnehmer.TeilnehmerID = Adresse.TeilnehmerID;
```

Gibt die Vornamen, Nachnamen und Straßennamen aller Teilnehmer zurück. **Teilnehmer ohne Adresse werden ebenfalls angezeigt**, aber mit NULL für die Straße.

Vorname	Nachname	Straße
Max	Mustermann	Hauptstraße
Anna	Schmidt	Nebenstraße
Tom	Müller	Bahnhofstraße
Lisa	Fischer	Rathausstraße
Paul	Wagner	Parkstraße
Julia	Becker	Lindenstraße

Vorname	Nachname	Straße
Felix	Hoffmann	Friedrichstraße
Sarah	Schulz	Goethestraße
David	Koch	Altstadtstraße
Laura	Bauer	Weserstraße
Michael	Weber	NULL

Wichtiger Hinweis: Beim INNER JOIN werden nur die Teilnehmer angezeigt, die eine Adresse in der Adress-Tabelle haben. Beim LEFT JOIN hingegen werden alle Teilnehmer angezeigt, unabhängig davon, ob sie eine Adresse haben oder nicht

Alias

```
SELECT Vorname AS Teilnehmer_Vornamen
FROM Teilnehmer;
```

Dieser SQL-Befehl gibt die Vornamen der Teilnehmer aus der Teilnehmer-Tabelle zurück und **benennt** die Spalte in Teilnehmer_Vornamen **um**.

Teilnehmer_Vornamen
Max
Anna
Tom
Lisa
Paul
Julia
Felix
Sarah
David
Laura
Michael

add noten zur Teilnehmern Tabelle

```
ALTER TABLE Teilnehmer
ADD COLUMN Note INT CHECK (Note >= 0 AND Note <= 100);
```

Dieser SQL-Befehl fügt eine neue Spalte namens Note zur Teilnehmer-Tabelle hinzu. Die Note-Spalte ist vom Datentyp INT und akzeptiert nur Werte im Bereich von 0 bis 100.

Add Noten jeweils Teilnehmer

```
UPDATE Teilnehmer
SET Note =
CASE
  WHEN TeilnehmerID = 1 THEN 85
  WHEN TeilnehmerID = 2 THEN 40
  WHEN TeilnehmerID = 3 THEN 48
  WHEN TeilnehmerID = 4 THEN 92
  WHEN TeilnehmerID = 5 THEN 85
  WHEN TeilnehmerID = 6 THEN 20
  WHEN TeilnehmerID = 7 THEN 78
  WHEN TeilnehmerID = 8 THEN 62
  WHEN TeilnehmerID = 9 THEN 58
  WHEN TeilnehmerID = 10 THEN 22
  WHEN TeilnehmerID = 11 THEN 48
END;
```

Dieser SQL-Befehl aktualisiert die Note-Spalte für jeden Teilnehmer basierend auf der TeilnehmerID. Jede TeilnehmerID erhält eine spezifische Note.

TeilnehmerID	Vorname	Nachname	Geburtsdatum	Email	Registrierungsdatum	Note
1	Max	Mustermann	1990-05-15	max.mustermann@example.com	2021-01-01	85
2	Anna	Schmidt	1985-08-22	anna.schmidt@example.com	2021-02-01	40
3	Tom	Müller	1995-03-10	tom.mueller@example.com	2021-03-01	48
4	Lisa	Fischer	1992-07-30	lisa.fischer@example.com	2021-04-01	92
5	Paul	Wagner	1988-11-12	paul.wagner@example.com	2021-05-01	85
6	Julia	Becker	1994-02-25	julia.becker@example.com	2021-06-01	20
7	Felix	Hoffmann	1991-09-18	felix.hoffmann@example.com	2021-07-01	78
8	Sarah	Schulz	1987-04-05	sarah.schulz@example.com	2021-08-01	62
9	David	Koch	1993-12-20	david.koch@example.com	2021-09-01	58
10	Laura	Bauer	1996-06-08	laura.bauer@example.com	2021-10-01	22
10	Michael	Weber	1998-12-05	michael.bauer@example.com	2021-10-01	48

Group by

```
SELECT Stadt, COUNT(TeilnehmerID) AS Anzahl_Teilnehmer
FROM Adresse
GROUP BY Stadt;
```

Dieser SQL-Befehl zählt die Anzahl der Teilnehmer pro Stadt und gruppiert das Ergebnis nach Stadt. **Erklärung:** COUNT(*) zählt die Anzahl der Zeilen (Teilnehmer) in jeder Stadt. GROUP BY Stadt gruppiert die Teilnehmer nach der Stadt.

Stadt	Anzahl_Teilnehmer
Berlin	4
Frankfurt	3
Bremen	3

```
SELECT a.Stadt, COUNT(t.TeilnehmerID) AS Anzahl_Bestandene_Teilnehmer
FROM Teilnehmer AS t
INNER JOIN Adresse AS a ON t.TeilnehmerID = a.TeilnehmerID
WHERE t.Note >= 50
GROUP BY a.Stadt;
```

Dieser SQL-Befehl zählt die Anzahl der bestandenen Teilnehmer (Note ≥ 50) pro Stadt und verwendet einen INNER JOIN, um nur Teilnehmer anzuzeigen, die eine zugehörige Adresse haben.

Stadt	Anzahl_Bestandene_Teilnehmer
Berlin	2
Frankfurt	2
Bremen	2

Hinweis: die alle Spalte, die nach Select anzeigt, muss nach Group BY Befehl geschrieben

Group by With having

```
SELECT a.Stadt, COUNT(t.TeilnehmerID) AS Anzahl_Bestandene_Teilnehmer
FROM Teilnehmer AS t
INNER JOIN Adresse AS a ON t.TeilnehmerID = a.TeilnehmerID
WHERE t.Note >= 50
GROUP BY a.Stadt;
HAVING COUNT(t.TeilnehmerID) > 2;
```

hier wird nur die städte angezeigt, die mehr als 2 passende Teilnehmer hat

Stadt	Anzahl_Bestandene_Teilnehmer
-------	------------------------------

Order By

```
SELECT t.Vorname, t.Nachname, t.Note
FROM Teilnehmer AS t
ORDER BY t.Note ASC;
```

Dieser SQL-Befehl gibt die Vorname, Nachname und Note der Teilnehmer aus, sortiert nach der **Note** in **aufsteigender** Reihenfolge.

Vorname	Nachname	Note
Julia	Becker	20
Tom	Müller	40
David	Koch	48
Michael	Weber	48
Laura	Bauer	58
Felix	Hoffmann	62
Sarah	Schulz	78

Vorname	Nachname	Note
Max	Mustermann	85
Paul	Wagner	85
Lisa	Fischer	92

```
SELECT t.Vorname, t.Nachname, t.Note
FROM Teilnehmer AS t
ORDER BY t.Nachname DESC, t.Vorname DESC;
```

Dieser SQL-Befehl gibt die Vorname, Nachname und Note der Teilnehmer aus, sortiert nach **Nachname und Vorname** in **absteigender** Reihenfolge.

Vorname	Nachname	Note
Michael	Weber	48
Paul	Wagner	85
Sarah	Schulz	78
Anna	Schmidt	40
Max	Mustermann	85
Tom	Müller	40
David	Koch	48
Felix	Hoffmann	62
Lisa	Fischer	92
Julia	Becker	20
Laura	Bauer	58

ORDER BY eith LIMIT

```
SELECT t.Vorname, t.Nachname, t.Note
FROM Teilnehmer AS t
ORDER BY t.Note ASC;
LIMIT 3
```

es ergibt nur die erste 3 ergebnisse, bzw die schlimmste 3 Noten.

Vorname	Nachname	Note
Julia	Becker	20
Tom	Müller	40
David	Koch	48

Funktionen

```
SELECT AVG(Note) AS Durchschnittsnote
FROM Teilnehmer;
```

Dieser SQL-Befehl berechnet den **Durchschnitt** der Noten aller Teilnehmer.

Durchschnittsnote

58

```
SELECT COUNT(TeilnehmerID) AS Anzahl_Teilnehmer
FROM Teilnehmer ;
```

Dieser SQL-Befehl zählt die **Anzahl** der Teilnehmer in der Teilnehmer-Tabelle.

Anzahl_Teilnehmer

11

```
SELECT SUM(Note) AS Gesamtnote
FROM Teilnehmer ;
```

Dieser SQL-Befehl berechnet die **Summe** aller Noten der Teilnehmer in der Teilnehmer-Tabelle und gibt das Ergebnis als Gesamtnote zurück.

Gesamtnote

638

```
SELECT MAX(Note) AS HöchsteNote
FROM Teilnehmer ;
```

Dieser SQL-Befehl gibt die **höchste** Note der Teilnehmer in der Teilnehmer-Tabelle zurück und benennt das Ergebnis als HöchsteNote.

HöchsteNote

92

```
SELECT MIN(Note) AS **NiedrigsteNote**
FROM Teilnehmer ;
```

Dieser SQL-Befehl gibt die niedrigste Note der Teilnehmer in der Teilnehmer-Tabelle zurück und benennt das Ergebnis als NiedrigsteNote.

NiedrigsteNote

20

```
SELECT DATE(Registrierungsdatum) AS Registrierungsdatum_ohne_Zeit
FROM Teilnehmer ;
```

Dieser SQL-Befehl gibt das Registrierungsdatum der Teilnehmer ohne die Zeitangabe zurück. Die Zeit wird entfernt, und es wird nur das Datum (Jahr, Monat, Tag) angezeigt.

Registrierungsdatum_ohne_Zeit
2023-05-01
2022-11-15
2021-03-22
2020-07-30
2021-12-15
2022-02-25
2021-09-18
2020-04-05
2021-12-20
2021-06-08

```
SELECT DAY(Geburtsdatum) AS Tag
FROM Teilnehmer ;
```

Der Befehl extrahiert den **Tag** des Geburtsdatums jedes Teilnehmers und gibt ihn in einer neuen Spalte mit dem Namen "Tag" zurück.

Tag
15
22
10
30
12
25
18
05
20
08
05

```
SELECT MONTH(Geburtsdatum) AS Monat
FROM Teilnehmer ;
```

es extrahiert den **Monat** des Geburtsdatums jedes Teilnehmers und gibt ihn in einer neuen Spalte mit dem Namen "Monat" zurück.

Monat
5
8
3
7
11
2
9
4
12
6
12

```
SELECT YEAR(Geburtsdatum) AS Jahr
FROM Teilnehmer ;
```

es extrahiert den **Jahr** des Geburtsdatums jedes Teilnehmers und gibt ihn in einer neuen Spalte mit dem Namen "Jahr" zurück.

Jahr
1990
1985
1995
1992
1988
1994
1991
1987
1993
1996
1998

```
SELECT CURDATE() AS Heute;
```

gibt das **aktuelle Datum** zurück

Heute
2025-11-06

```
SELECT NOW() AS Heute;
```

gibt das **aktuelle Datum mit Uhrzeit** zurück

Heute
2025-11-06 14:37:58

```
SELECT WEEKDAY(Geburtsdatum) AS Wochentag
FROM Teilnehmer ;
```

es gibt den Wochentag (als Zahl) für jedes Geburtsdatum der Teilnehmer zurück, wobei 0 für Montag, 1 für Dienstag, und so weiter bis 6 für Sonntag steht.

Wochentag
0
1
3
5
2
5
6
0
0
2
6

```
SELECT DAYNAME(Geburtsdatum) AS Wochentag_Name
FROM Teilnehmer ;
```

es gibt den Namen des Wochentages (z.B. Montag, Dienstag, ...) für jedes Geburtsdatum der Teilnehmer zurück.

Wochentag_Name
Monday
Tuesday
Sunday
Friday
Wednesday
Friday
Saturday
Monday

Wochentag_Name
Monday
Wednesday
Saturday

```
SELECT Datediff(year,Geburtsdatum, Now()) AS Alter_Jahren from Teilnehmer ;
```

es ergibt den Unterschied zwischen zwei Datums je nach die gegebene Date_Element (Jahr, Monat, Tag)
Datediff(Date_Element,Startdate, Enddate)

Alter_Jahren
20
22
34
34
34
33
41
17
20
19
28

Operatoren

```
SELECT Vorname, Nachname
FROM Teilnehmer
WHERE Geburtsdatum > '1990-01-01' AND Note >= 50;
```

gibt die Vornamen und Nachnamen der Teilnehmer zurück, deren Geburtsdatum nach dem 1. Januar 1990 liegt und deren Note mindestens 50 beträgt.

Vorname	Nachname
Max	Mustermann
Lisa	Fischer
Felix	Hoffmann
David	Koch

```
SELECT t.Vorname, t.Nachname
FROM Teilnehmer AS t
```

```
INNER JOIN Adresse AS a ON t.TeilnehmerID = a.TeilnehmerID
WHERE t.Note < 50 OR a.Stadt = 'Frankfurt';
```

Diese Abfrage gibt alle Teilnehmer Vor und Nachnamen mit einer Note unter 50 oder mit der Stadt 'Frankfurt' in der Adresse zurück

Vorname	Nachname
Anna	Schmidt
Tom	Müller
Lisa	Fischer
Paul	Wagner
Julia	Becker
Laura	Bauer

```
SELECT t.Vorname, t.Nachname
FROM Teilnehmer AS t
INNER JOIN Adresse AS a ON t.TeilnehmerID = a.TeilnehmerID
WHERE a.Stadt <> 'Frankfurt';
```

Dieser SQL-Befehl gibt die Vornamen und Nachnamen der Teilnehmer zurück, deren Stadt nicht 'Frankfurt' ist

Vorname	Nachname
Max	Mustermann
Anna	Schmidt
Tom	Müller
Julia	Becker
Felix	Hoffmann
Sarah	Schulz
David	Koch
Laura	Bauer

```
SELECT t.Vorname, t.Nachname
FROM Teilnehmer AS t
INNER JOIN Adresse AS a ON t.TeilnehmerID = a.TeilnehmerID
WHERE (t.Note >= 50 AND a.Stadt = 'Berlin') OR (t.Note < 50 AND a.Stadt = 'Frankfurt');
```

Es zeigt die Teilnehmer, die entweder eine Note von 50 oder mehr in Berlin haben oder eine Note von weniger als 50 in Frankfurt.

Vorname	Nachname
Max	Mustermann
Felix	Hoffmann
Julia	Becker

String Funktionen

```
SELECT Vorname, Nachname
FROM Teilnehmer
WHERE Vorname LIKE 'A%';
```

Es sucht alle Teilnehmer, deren Vorname mit dem Buchstaben "A" **beginnt**.

Vorname	Nachname
Anna	Schmidt

```
SELECT Vorname, Nachname
FROM Teilnehmer
WHERE Vorname LIKE 'A_';
```

Es sucht nach Teilnehmern, deren Vorname mit dem Buchstaben "A" **beginnt** und **nur** ein weiteres Zeichen danach enthält.

Vorname	Nachname
---------	----------

```
SELECT Vorname, Nachname
FROM Teilnehmer
WHERE Vorname LIKE '%a%';
```

sucht nach Vornamen, die den Buchstaben "a" an **beliebiger** Stelle enthalten.

Vorname	Nachname
Max	Mustermann
Anna	Schmidt
Lisa	Fischer
Paul	Wagner
Julia	Becker
Sarah	Schulz
David	Koch
Laura	Bauer
Michael	Weber

```
SELECT Vorname, Nachname
FROM Teilnehmer
WHERE Vorname LIKE '_a%';
```

sucht nach Vornamen, bei denen der **zweite Buchstabe** ein "a" ist.

Vorname	Nachname
---------	----------

Vorname	Nachname
Max	Mustermann
Paul	Wagner
Sarah	Schulz
David	Koch
Laura	Bauer

```
SELECT Vorname, Nachname
FROM Teilnehmer
WHERE left(Vorname,2)='an';
```

erste zwei Buchstaben sollen an sein

Vorname	Nachname
Anna	Schmidt

```
SELECT Vorname, Nachname
FROM Teilnehmer
WHERE right(Vorname,3)='aul';
```

letzte drei Buchstaben sollen aul sein

Vorname	Nachname
Paul	Wagner

Tabelle Löschen

```
DROP TABLE Teilnehmer;
```

User und Berechtigungen

```
CREATE USER Timo IDENTIFIED BY '12345';
```

Der User Timo wird erstellt mit eine Passwort : '12345'

```
GRANT SELECT on Teilnehmer TO Timo;
```

Timo hat lesen Recht für die Tabelle Teilnehmer erhalten.

```
Revoke SELECT on Teilnehmer FROM Timo;
```

das Lesen Recht wird von timo entzogen

```
CREATE ROLE schreib_recht IDENTIFIED BY '12345ggg';  
GRant UPDATE,INSERT,DELETE on Teilnehmer TO schreib_recht ;
```

Die Rolle Variable schreib_recht wird erstellt Die UPDATE,INSERT,DELETE Rechte von Tabelle Teilnehmer werden in dem Variable schreib_recht gespeichert

```
GRANT schreib_recht TO Timo;
```

Die alle Rechte, die in Variable schreib_recht gespeichert sind

```
REVOKE DELETE,UPDATE FROM schreib_recht;
```

Die DELETE,UPDATE Rechte werden von schreib_recht ausgezogen bzw von Timo
